EdaphNet

7 DAYS × 2 HOURS · STUDY GUIDE

What every file does · What is inside it · How it works · How the files connect
Daily reading · Code exercises · Self-tests

Prepared for: [Your name] · Register number: [Roll number]
Guide: [Guide name], [Department]

Version 1.0 · 3 October 2026 · companion to the Complete Project Dossier

Contents

The map you are going to learn

Day 1 — The big picture (what, why, and seeing it work)

Day 2 — The data: from raw files to a dataset

Day 3 — The witness: what colour knows about soil

Day 4 — Errors and swaps: what can go wrong in a report

Day 5 — The method: how EdaphNet scores a batch

Day 6 — Experiments and statistics: how a result is produced

Day 7 — Results, the paper, and defending it

Appendix A — Which file imports which (the links)

Appendix B — Files you can skip for the paper

Appendix C — When you are stuck

Every entry is a clickable link. Day pages start on a new page, so you can print one day at a time.

This guide teaches you the whole project: what every file does, what is inside it, how it works, and how the files connect. Follow it in order. Each day has the same shape:

Part	Time	What you do
Read	30 min	the documents for the day (they explain the ideas)
Code	60 min	open the files, read the parts named below, run the exercises
Write	30 min	answer the self-test in a notebook, in your own words, without looking

Setup (once, 10 minutes)

- Open the folder `D:\MY DATA\Downloads\new_to_do-main\edaphnet-v2` in VS Code.
- Open a terminal in VS Code (Ctrl + `) and set the Python you will use:

```
$PY = "C:\Program Files\Python314\python.exe"
```

- Useful VS Code keys: **Ctrl+Click** on a function name jumps to where it is defined; **Ctrl+Shift+F** searches all files; **Ctrl+Shift+V** shows a Markdown file nicely.
- Rules for reading code: read the **docstring at the top** of each file first, then the **function names**, then only the functions named in this guide. Never read a file top to bottom.
- To try code, start Python in the project folder (`& $PY`) and paste the exercise snippets.

The map you are going to learn

```
                ┌──────────────────────── DATA ────────────────────────┐
download_ossl.py → data/raw/ossl/*.csv.gz → build_ossl_dataset.py → data/processed/t1_kssl3.npz
                                    (real_data.py: cleaning, colour from spectra)  │
                                                                                   ▼
                                   fit_witness_ladder.py → data/processed/witness_kssl3.npz
                                   (LightGBM witness models: predictions μ and covariance Σ)
                                                                                   │
                ┌──────────────────────── EXPERIMENT ──────────────────────────────▼──────┐
configs/real_kssl3.yaml → run_real.py ── plants errors (operators.py, swaps.py)
                                    ── scores batches: inference.py  (the method)
                                                       baselines.py  (B0–B7)
                                    ── p-values + BH: conformal.py ; metrics: metrics.py
                                    ── run record: tracking.py → results/runs/<run>/
                ┌──────────────────────── ANALYSIS & WRITING ──────────────────────────┐
results/runs/<run>/predictions.csv.gz → analyse_amendment.py (verdict) → docs/21
                                    → exploratory_tables.py, rescore_pools.py,
                                      diag_gross.py, error_mix.py → results/exploratory/ → docs/23, 26
                                    → make_paper_figures.py → docs/paper/figures/ → docs/paper/draft_v0.md
```

`space.py` (how values are stored and printed) is used by almost everything. `simulate.py` and `properties.py` replace the DATA box for the synthetic experiment (`run_sim.py`).

Day 1 — The big picture (what, why, and seeing it work)

Goal: explain in 2 minutes what the project does and why, and run it.

Read (30 min)

1. `reports/EdaphNet_Project_Dossier.pdf`, sections 1–4 (at a glance, problem, objectives, scope).
2. `README.md`, sections 1–4 (the problem in simple words, progress, findings).

Code and run (60 min)

1. Run the tests, so you know the code works on your laptop:

   ```
   & $PY -m pytest -q
   ```

   Expect `34 passed, 1 deselected`.

2. Run the command-line demo and read what it prints:

   ```
   & $PY scripts/demo.py
   & $PY scripts/demo.py --witness W3
   ```

   It builds one batch of 24 real KSSL reports, plants a swap and a ×10 error, runs the method, and prints the re-test list. Compare the two runs: colour witness vs spectrometer.

3. Run the live demo and press **Guided demo**:

   ```
   & $PY scripts/serve_demo.py
   ```

   Open http://localhost:8000. Watch the four stages. Then click "Make N ×10" and "Report of a different soil" yourself and press Run check.

4. Open `scripts/demo.py` and read only `main()`. Notice the imports at the top: they tell you which `src/edaphnet2` modules the demo uses. Write them down; you will meet each one this week.

How the files connect today: `demo.py` and `serve_demo.py` both call `run_real.load()` to get the data and `inference.infer_batch()` to score a batch. They are thin front ends over the same method.

Self-test

1. What are the two kinds of errors? Give one example of each.
2. What is a "witness"? Why can't the laboratory corrupt it?
3. Why does a lab need a *ranked list* instead of checking every report?
4. In the demo, which error did colour catch, and which did the spectrometer catch better?

Day 2 — The data: from raw files to a dataset

Goal: know where the soil data comes from, what is real, and how it becomes `t1_kssl3.npz`.

Read (30 min)

1. Dossier section 9 (datasets) and 9.4 (what is real and what is simulated).
2. `docs/12_data_audit_ossl.md`: skim; read §7 (the counting mistake and its correction).

Code (60 min)

1. `scripts/download_ossl.py`: read the docstring and `main()`. It downloads three files and checks each one's MD5 fingerprint against `data/ossl_manifest_md5.txt`. Why fingerprints? So you know the data is exactly the data the paper used.
2. `scripts/build_ossl_dataset.py`: read the docstring and the `argparse` lines. It is run twice: `--dataset KSSL.SSL --props kssl3` and `--dataset ICRAF.ISRIC --props icraf8`.
3. `src/edaphnet2/real_data.py`, read these:
   - `PROPERTIES` and `PROPERTY_SETS` (near the top): which properties exist, which set each dataset uses.
   - `physical_screen()`: removes impossible values (e.g. negative carbon).
   - `class LatentTransform` with `to_latent()` and `to_natural()`: puts each property on a standard scale (log for carbon, which spans 0.1% to 50%) and back.
4. `src/edaphnet2/space.py`: read `class Space`, `to_latent()`, `print_values()`. `print_values()` rounds numbers the way a lab report prints them; this matters because a "swapped digits" error happens on the *printed* number.
5. **Exercise**: look at the real dataset.

```
import numpy as np, collections
d = np.load("data/processed/t1_kssl3.npz", allow_pickle=True)
print(d.files)                        # what is stored
print(d["names"], d["X"].shape)       # ['OC' 'N' 'Ctot'] (19746, 3)
print(d["X"][:3])                     # first three reports: OC %, N %, total C %
print(collections.Counter(d["split"].astype(str)))   # train 9873, cal 2924, test 6949
```

   `X` = laboratory values, `full` = spectra, `lab` = colour (CIELAB), `groups` = location (used for splitting), `split` = train / calibration / test.

How the files connect today: `download_ossl.py` → `data/raw/ossl/` → `build_ossl_dataset.py` (using `real_data.py`) → `data/processed/t1_*.npz` + `data/splits/*.csv.gz`.

Self-test

1. Which parts of the data are real, and which are simulated?
2. Why are train and test split by *location* and not at random?
3. What does `to_latent()` do, and why use a log scale for carbon?
4. What was the counting mistake in the data audit, and how was it caught?

Day 3 — The witness: what colour knows about soil

Goal: understand how colour is computed from a spectrum, how a witness model predicts soil properties, and why colour mainly knows carbon.

Read (30 min)

1. `docs/14_step4b_witness_ladder.md`: the witness ladder and the R^2 table.
2. Dossier sections 10.1 (witness model) and 14.1 (Figure 4).

Code (60 min)

1. `src/edaphnet2/real_data.py`: read `spectra_to_lab()`. It converts a laboratory spectrum to the colour a camera would see (CIELAB, D65 light). `coarse_bands()` gives the 10-band witness. These are called during the dataset build (`build_ossl_dataset.py`), which stores colour as `lab` and bands as `bands` in `t1_kssl3.npz`.
2. `scripts/fit_witness_ladder.py`: read the docstring, `fit_predict()` (one LightGBM model per property) and find where it adds phone-like noise (ΔE 0, 2, 4, 8). It predicts each sample's properties out-of-fold, takes the covariance of the prediction errors as Σ, and writes `witness_kssl3.npz`: for each rung, `__mu` (predictions) and `__S` (covariance).
3. `src/edaphnet2/witness.py` (57 lines, read all of it). This is how the experiments *use* μ and Σ:
   - `GaussianModel.fit()`: learns a mean and covariance (used by the synthetic experiment).
   - `GaussianModel.logpdf_latent()`: how likely a report is, given the witness prediction μ.
   - `FixedGaussianModel`: the same with Σ taken from the witness file (used by `run_real.py` and the demos).
4. **Exercise**: open the witness file.

```
import numpy as np
w = np.load("data/processed/witness_kssl3.npz", allow_pickle=True)
print(w.files)          # W0__mu, W0__S, W1_dE0__mu, W1_dE2__mu, ..., W3__mu
print(w["W1_dE2__S"])   # 3×3 covariance: how OC, N, total C errors move together
```

5. Open `results/runs/20261001-234621_witness_ladder_kssl3_8148163/witness_ladder_metrics.csv` in Excel or VS Code. Find R^2 for OC with W1_dE2 (≈ 0.70) and with W3 (spectrometer).

How the files connect today: `build_ossl_dataset.py` (with `real_data.spectra_to_lab`) stores colour in `t1_kssl3.npz` → `fit_witness_ladder.py` (LightGBM) → `witness_kssl3.npz` → read by `run_real.load()` → turned into a `witness.FixedGaussianModel` for scoring.

Self-test

1. Why does colour predict organic carbon well but pH or potassium poorly?
2. What is μ and what is Σ in the witness model? Why does a *full* Σ help find a single wrong value?
3. What does "ΔE 2" mean, and why is calculated colour a *best case* for a phone?
4. What is R^2, and why is it measured on unseen locations?

Day 4 — Errors and swaps: what can go wrong in a report

Goal: know every error type, how errors are planted, and how swap patterns are listed.

Read (30 min)

1. `docs/11_problem_specification.md`: §2.3 (swap prior) and §2.4 (corruption library).
2. Dossier sections 10.2, 10.3 and 10.7.

Code (60 min)

1. `src/edaphnet2/operators.py`:
   - `GROUPS` (top): decimal, unit, transposition, column_swap, gross.
   - `transpose_candidates(value, decimals)`: all numbers made by swapping two adjacent digits.
   - `gross_grid()`: the grid of multiplicative factors for gross errors.
   - `enumerate_hypotheses()`: for one report, every way it *could* have been corrupted, with prior probabilities. This is the list the method checks.
   - `apply_random_corruption()`: plants one random error (used to make test data). 2. `src/edaphnet2/swaps.py` (74 lines, read all):
   - `enumerate_swaps(n)`: S1 neighbour swaps, S2 tray rotations (shift by 1 or 2), S3 any pair.
   - `log_prior()`: the structured prior over swap classes.
   - `random_swap()`: plants one swap event. 3. `src/edaphnet2/simulate.py`: skim `Generator.batch()`. Synthetic soils with a witness of chosen strength λ, used only for the first experiment.
2. **Exercise**:

```
import sys; sys.path.insert(0, "src")
import numpy as np
from edaphnet2 import swaps, operators
h = swaps.enumerate_swaps(6)
print(len(h), h[:2])          # 41 swap hypotheses for a batch of 6
print(swaps.random_swap(6, np.random.default_rng(1)))   # e.g. ('S2', [3 4 0 1 2 5]) = tray rotation
print(operators.transpose_candidates(0.16, 2))          # [0.61]
```

   Now try `enumerate_swaps(24)` and print `len(...)`: this is how many swap patterns the method scores for every real batch.
3. Read `tests/test_core.py`: `test_transposition_is_involution` and `test_swap_hypotheses_are_permutations`. Tests are the best examples of how to use a function.

How the files connect today: `run_real.make_batch()` uses `operators.apply_random_corruption()` and `swaps.random_swap()` to plant errors; `inference.py` uses `operators.enumerate_hypotheses()` and `swaps.enumerate_swaps()` to search for them.

Self-test

1. List the five value-error groups with an example each.
2. What are S1, S2 and S3? Which one models "the whole tray shifted by one place"?
3. What is the difference between *planting* an error and *hypothesising* an error?
4. Why is the "structured prior" hypothesis (H6) reported as not supported?

Day 5 — The method: how EdaphNet scores a batch

Goal: explain step by step how `infer_batch()` turns a batch into scores, explanations and corrections, and how the baselines differ.

Read (30 min)

1. Dossier section 10.4 (joint inference, the two equations) and 11 (models, ablations, baselines).
2. `docs/paper/draft_v0.md` §4 Method.

Code (60 min)

1. `src/edaphnet2/inference.py` (129 lines, the heart of the project; read all of it slowly):
   - `class InferenceConfig`: settings (ρ = swap rate, ε = error rate, which parts are switched on).
   - `report_hypotheses()`: for each report, the list of possible corruptions (from `operators`).
   - `loglik_matrix()`: ℓ(j, i) = how well report j fits sample i's witness, allowing any corruption. This is a matrix: reports × samples.
   - `infer_batch()`: scores every swap hypothesis with log prior + sum of ℓ changes, normalises, then for each report computes P(moved), P(corrupted), the suspicion score, the explanation (`kind`, `partner`, `value_op`) and the correction.
   - `loglik_none()`: the same matrix with no corruption allowed (used by the swap-matcher baseline). Draw the flow on paper: report values + witness μ → ℓ matrix → swap posteriors → per-report score.
2. `src/edaphnet2/baselines.py`: for each baseline, read only its docstring: `RangeCheck` (B0, also B6 repair via `attribute()`), `rule_score` (B1), `population_score` (B2), `marginal_witness_score` (B3), `hungarian_score` (B4), `RegressionVeracity` (a simplified B7 used only on synthetic data), `CleanlabOutre` (B7 on real data). B5 (supervised LightGBM) lives in `scripts/run_real.py` → `run_setting()`.
3. Run the two method tests and read them:

   ```
   & $PY -m pytest tests/test_core.py -v -k "decimal_error or swap_detected"
   ```

   `test_decimal_error_detected_and_attributed` plants a ×10 error and checks the method finds it and names it; `test_swap_detected_with_strong_witness` does the same for a swap.
4. Re-run `scripts/demo.py` and match each printed column to a field returned by `infer_batch()`.

How the files connect today: `inference.py` imports `operators`, `swaps`, `space`; it receives μ from `witness_*.npz` and a `FixedGaussianModel` from `witness.py`. `baselines.py` imports `operators` and `space`. `run_real.score_batch()` calls it for every batch, together with every baseline.

Self-test

1. In your own words: what does ℓ(j, i) measure?
2. How does the method decide that report 4 and report 9 were swapped?
3. What is the suspicion score? What is the explanation? What is the correction?
4. Which baseline is the strongest, and what unfair advantage does it have?
5. Why can the method name the error type when B5 cannot?

Day 6 — Experiments and statistics: how a result is produced

Goal: understand how an experiment runs, how "errors found at a 5% budget" and false-alarm control are computed, and the calibration mistake.

Read (30 min)

1. Dossier sections 12 (experiments), 13 (metrics) and 14.4 (calibration mistake).
2. `docs/decision_log.md` : entries D-012, D-016, D-017.

Code (60 min)

1. `configs/real_kssl3.yaml` : every setting of the main test (seeds, batch size 24, pools of 50, witness rungs, orderings, error rates). Change nothing; just read it.
2. `scripts/run_real.py` , read in this order:
   - `load()` : loads the dataset and witness files.
   - `sample_indices()` : picks samples for a batch, random or clustered.
   - `make_batch()` : plants errors (Day 4).
   - `score_batch()` : runs the method, its ablations and the baselines on one batch (Day 5).
   - `run_setting()` : one full setting: calibration batches, B5 training, test batches, p-values, metrics. Find the line with `recall_at_budget` .
   - `main()` and `prepare_resume()` : loops over settings; resumes safely if interrupted. 3. `src/edaphnet2/conformal.py` (28 lines): `conformal_pvalues()` and `benjamini_hochberg()` . It is used through `metrics.py` .
3. `src/edaphnet2/metrics.py` : `recall_at_budget()` (the primary metric), `detection_metrics()` , `weighted_conformal_pvalues()` , `mondrian_pvalues()` .
4. `src/edaphnet2/tracking.py` : `provenance()` records the code version (git commit) and marks a run "clean" or "exploratory"; `new_run_dir()` names the run folder.
5. **Exercise** (false-alarm control on toy numbers):

```
import sys; sys.path.insert(0, "src")
import numpy as np
from edaphnet2.conformal import conformal_pvalues, benjamini_hochberg
from edaphnet2.metrics import recall_at_budget
rng = np.random.default_rng(0)
cal  = rng.normal(size=1000)                                  # scores of 1,000 clean calibration reports
test = np.r_[rng.normal(size=95), np.full(5, 4.0)]          # 95 clean + 5 erroneous test reports
p = conformal_pvalues(cal, test)
print(p[-5:])                                    # the 5 errors get p = 0.001 = 1/(1000+1), the smallest poss
print(benjamini_hochberg(p, 0.10).sum())       # 7 flagged: 5 errors + 2 false alarms
scores = np.r_[rng.random(95), np.full(5, 2.0)]
is_err = np.r_[np.zeros(95, bool), np.ones(5, bool)]
print(recall_at_budget(scores, is_err, np.zeros(100, int), 0.05, 0)[0])   # 1.0: all 5 errors in the top 5%
```

   Then try the **calibration mistake**: replace `cal` with `np.repeat(cal[:50], 20)` (50 samples counted 20 times each) and see how the p-values and the number flagged change.
6. Optional quick run (a few minutes): `& $PY scripts/run_real.py --config configs/real_kssl3.yaml --smoke` and look at the new folder in `results/runs/` .

How the files connect today: `configs/*.yaml` → `run_real.py` (uses `inference`, `baselines`, `conformal`, `metrics`, `tracking`) → `results/runs/<run>/` with `config.json`, `provenance.json`, `metrics_by_setting.csv`, `predictions.csv.gz`.

Self-test

1. What exactly is "recall at a 5% budget"? Why per pool of 50 batches?
2. What is a conformal p-value? What is the smallest value it can take?
3. What did the calibration mistake do, and how was it fixed?
4. Why can BH over a pool of 1,200 reports flag almost nothing, while per batch it works?
5. What makes a run record trustworthy (three things)?

Day 7 — Results, the paper, and defending it

Goal: know every main result, where it comes from, and answer the hard questions.

Read (30 min)

1. `docs/18_preregistered_amendment_v2.md` §2 (the rules, written *before* the test) and `docs/21_kssl_preregistered_results.md` (the verdict).
2. `docs/26_review_fixes.md` (gross errors, error mix, five seeds).

Code (60 min)

1. `scripts/analyse_amendment.py`: read the docstring. It applies the pre-registered rules to the run and writes `amendment_A1.csv`, `amendment_verdict.json`. Open both in `results/runs/20261001-234902_real_kssl3_095deaa/`.
2. Skim the docstrings of the exploratory scripts: `exploratory_tables.py` (swaps vs values, ablations), `rescore_pools.py` (pool size), `diag_gross.py` (gross errors), `error_mix.py` (error mix). Open their CSV outputs in `results/exploratory/20261001-234902_real_kssl3_095deaa/`.
3. `scripts/make_paper_figures.py`: see that every figure reads a CSV from a run folder; no number is typed by hand. Open the six PNGs in `docs/paper/figures/`.
4. Read the paper `docs/paper/draft_v0.md`: Abstract, §7 (results), §8 (discussion and limitations).
5. **Trace one number yourself** (the most useful skill for the viva): pick "+0.092 with a spectrometer" and follow it: paper §7.4 → doc 21 → `amendment_A1.csv` → `run_real.py` → `inference.py`. The same path is written out in `reports/EdaphNet_Files_Explained.pdf` §2.

Self-test (say the answers aloud, as if to your guide)

1. Explain the project in 2 minutes.
2. Is your data real? (Real soils and lab values; planted errors; calculated colour.)
3. Why did the main claim fail? (Colour finds more swaps, misses more wrong values; they cancel at a 50/50 mix.)
4. What is new? (Joint checking with an independent witness; explanation 97–100%; the error-mix map; the calibration finding.)
5. What would you do next? (Real photos, real errors, Indian soils; register amendment v3.)
6. What were the mistakes, and what did you learn from them?

Appendix A — Which file imports which (the links)

File	Imports / uses	Used by
`space.py`	—	`operators.py`, `baselines.py`, `inference.py`, `run_real.py`
`properties.py`	—	`simulate.py`, `witness.py`, `run_sim.py`
`operators.py`	`space`	`inference.py`, `baselines.py`, `simulate.py`, `run_real.py`
`swaps.py`	—	`inference.py`, `simulate.py`, `run_real.py`
`witness.py`	`properties`	`run_sim.py` (GaussianModel); `run_real.py`, `demo.py`, `serve_demo.py` (FixedGaussianModel)
`inference.py`	`operators`, `swaps`, `space`	`run_sim.py`, `run_real.py`, `demo.py`, `serve_demo.py`
`baselines.py`	`operators`, `space`	`run_sim.py`, `run_real.py`, `demo.py`
`conformal.py`	—	`metrics.py` (and the tests)
`metrics.py`	`conformal`	`run_sim.py`, `run_real.py`
`real_data.py`	—	`build_ossl_dataset.py`, `fit_witness_ladder.py`, `run_real.py`
`simulate.py`	`operators`, `swaps`, `properties`	`run_sim.py`
`tracking.py`	—	`run_sim.py`, `run_real.py`, `fit_witness_ladder.py`
`phone.py`	—	`process_photos.py`, `serve_demo.py`

`rescore_pools.py` and the other analysis scripts read the saved CSV files directly and do not import the method.

(Exact import lines are at the top of each file; Ctrl+Click any name to jump to it.)

Appendix B — Files you can skip for the paper

- `app/`, `scripts/app_*.py`, `data/app_raw/`, `docs/24`, `docs/25`: the separate farmer app.
- `phone.py`, `process_photos.py`, `templates/`, `docs/15`: for paper 2 (real photos).
- `demo_site/`, `serve_demo.py`, `build_demo_page.py`, `train_colour_brain.py`: demonstrators.

Learn them later if you want; they do not affect any number in the paper.

Appendix C — When you are stuck

1. Read the file's docstring again; most files explain themselves in the first 10 lines.
2. Find a test that uses the function (`Ctrl+Shift+F` in `tests/`).
3. Look the topic up in the glossary (Dossier Appendix B) and in `FILE_GUIDE.md`.
4. Write the question down and bring it to your guide.